Standar Penulisan Kode

Versi PHP

Developer Dinas Komunikasi dan Informatika
Kabupaten Badung.

Daftar Isi

Daftar Isi **1**

Environment **2**

Scope **2**

Penulisan Kode **3**

- Struktur File dan Folder 3
- Penamaan File, Class, Function dan Variabel 3
- Laravel Admin 5
- Pola Service - Repository 5
- Pola Service pada Laravel Admin 9
- View Resources 10
- Helpers 14
- Deployment 16
- Tag on GIT Messages 17
- Versioning 18

Environment

Standar environment yang digunakan yaitu:



- **Local** merupakan environment yang dijalankan pada perangkat keras developer
- **Demo** merupakan environment yang dijalankan pada server khusus development untuk functional testing
- **Live** merupakan aplikasi produksi (production) dengan environment yang dijalankan pada server produksi

Scope

Ruang lingkup penulisan kode program yaitu

- Framework Laravel
- MySQL / MariaDB
- Laravel Admin (dependency)
- Helpers Register (dependency)
- Git, dengan repo pada Gitlab Badung
- Docker

Penulisan Kode

1. Struktur File dan Folder

Struktur file dan folder paling tidak harus meliputi:

```
app/
  Admin/ ----------------------> Admin
    Controllers/  -------------> Admin Controllers
      API/ --------------------> Admin API Controllers
        AdminApiController.php
      AdminController.php
    route.php -----------------> Admin route (including Admin API)
  Http/ -----------------------> Public
    Controllers/ --------------> Public Controllers
      API/ --------------------> Public API Controllers
        PublicApiController.php
       PublicController.php
  Helpers/ --------------------> Helpers class & functions
    Methods/ ------------------> Helpers berupa functions
      formatter.php
    AbcHelper.php
  Repositories/ ---------------> Repositories
    EntityRepository.php
  Services/ -------------------> Services
    EntityService.php
  ModelEloquent.php
routes/
  api.php ---------------------> Public routes
  web.php ---------------------> Public API routes
```

2. Penamaan File, Class, Function dan Variabel

➔ Penamaan class menggunakan **Pascal Case**, yaitu tanpa spasi dan kata paling awal dan kata selanjutnya diawali huruf besar.

```
ArticleController
Article (eloquent)
ArticleHelper
```

➔ Penamaan function menggunakan **Camel Case**, yaitu tanpa spasi dan kata paling awal menggunakan huruf kecil dan kata selanjutnya diawali huruf besar.

```
doCalculate
getArticles
setTime
```

➔ Penamaan variabel pada **PHP** dan **JS** menggunakan **Snake Case**, yaitu semua spasi dengan garis bawah atau underscore dengan semua huruf kecil atau lowercase.

```
$article
$dev_badung
let article;
let dev_badung;
```

➔ Penamaan helper berupa class memiliki **suffix (akhiran) Helper** dan menggunakan **Pascal Case**, yaitu tanpa spasi dan kata paling awal dan kata selanjutnya diawali huruf besar.

```
IndonesiaTimeHelper
ArrayHelper
```

➔ Penamaan helper berupa function memiliki **suffix (akhiran) _helper** dan menggunakan **Snake Case**, yaitu semua spasi dengan garis bawah atau underscore dengan semua huruf kecil atau lowercase.

```
time_helper
array_helper
```

3. Laravel Admin

Penggunaan laravel admin secara umum dapat dilihat pada https://laravel-admin.org/docs/en/

Dalam hal pengembangan aplikasi yang komplek (bukan CRUD) dan beragam pola, maka disarankan untuk menggunakan pengkodean yang ditulis mandiri (tidak menggunakan pola/builder dari laravel admin. Untuk hal tersedia, penggunaan customized view akan menjadi

```php
use Encore\Admin\Layout\Content;
use Illuminate\Routing\Controller;

class AdminController extends Controller
{
   public function index(Content $content)
   {
       return $content
           ->title('Title')
           ->description('Sub title')
           ->view('view', compact('data', 'data'));
   }
}
```

4. Pola Service - Repository

- **Models (Eloquent)**, menyediakan objek yang merangkum tabel database, kolom, hubungan, dan lain-lain.

```
Siswa
MataPelajaran
Rapor
```

➔ **Repositories**, menyediakan akses data dan enkapsulasi persisten untuk implementasi tertentu yang memungkinkan pertukaran data yang mudah. Repositories berisi models dan queries.

```
SiswaRepository
      getAll
      insert
      setStatus
MataPelajaranRepository
      get
      getAllBySiswa
RaporRepository
      insert
```

➔ **Services**, menyediakan logika bisnis dan algoritma untuk proses tertentu. Services dapat menggunakan berbagai repositori dan service lainnya untuk mengimplementasikan logika kompleks.

```
SiswaService
      register
            SiswaRepository->insert
      acceptRegisterSiswa
            SiswaRepository->setStatus
RaporService
      saveNilaiRapor
            MataPelajaranRepository->get
            SiswaRepository->getAll
            foreach($students as $student)
                  RaporRepository->insert
```

Service juga dapat digunakan pada controller lain dengan logika bisnis yang sama sehingga dapat mencegah penulisan kode yang berulang dan menjaga konsistensi logika bisnis.
Contoh:

- Modul pendaftaran siswa pada web dan api. Pada contoh kasus ini service yang sama yaitu register siswa akan digunakan pada 2 controllers yaitu untuk web dan api.
- Modul buat surat dan buat multiple surat. Pada contoh kasus ini service buat surat digunakan pada 2 controllers .

➔ **Controllers**, menerima permintaan, meneruskan data ke service yang sesuai, dan mengembalikan data yang diproses sebagai respon kepada pengguna.

```
SiswaController
      Register
            $request->validate
            SiswaServices->register
            return response
RaporController
      saveNilai
            $request->validate
            RaporService->saveNilaiRapor
            Return response
```

Perbedaan Service dan Repository

Service	Repository
- Berisi logika bisnis berupa algoritma untuk suatu proses tertentu - Setiap fungsi representasi sebuah proses logika bisnis / algoritma proses komputasi - 1 fungsi service dapat menggunakan 1/lebih repository - Contoh: 1. Pendaftaran Siswa 2. Get Data Siswa 3. Set Siswa Lulus 4. Set Siswa DO	- Berisi logika akses dan manipulasi data melalui model / tabel dan query - Setiap fungsi representasi sebuah logika akses / manipulasi data - 1 fungsi repository merupakan operasi / perintah untuk akses / manipulasi data di database - Contoh: 1. Insert Siswa 2. Get Siswa 3. Update Status Siswa

Contoh Kasus Penggunaan Pola Service - Repository

Controller

```php
class PendaftaranSiswaController extends Controller
{
    public function register(Request $req, SiswaService $siswaServ)
    {
        $req->validate([
            'nama' => ['required'],
            'email' => ['required'],
        ]);

        $siswaServ->register(
            $request->get('name'),
            $request->get('email')
        );

        return response()->redirect('home');
    }
}
```

Service

```php
class SiswaService
{
    function register(
        $name, $email,
        SiswaRepository $siswaRepo,
        PembayaranRepository $paymentRepo
    )
    {
        $siswa = $siswaRepo->insert($name, $email);
        $paymentRepo->paymentSiswa($siswa, 30000);
    }
}
```

Repositories

```
> SiswaRepository.php
class SiswaRepository
{
    function insert($name, $email)
    {
        $siswa = new Siswa; // Model
        $siswa->name = $name;
        $siswa->email = $email;
        return $siswa->save();
    }
}

> PembayaranRepository.php
class PembayaranRepository
{
    function paymentSiswa($siswa, $nominal)
    {
        $payment = new Pembayaran(['nominal' => $nominal]);
        return $siswa->payments()->save($payment);
    }
}
```

5. Pola Service pada Laravel Admin

Pola ini merupakan adaptasi pola Service-Repository pada poin Pola Service - Repository, yang mana penerapannya akan berbeda saat penggunaan laravel-admin.

Ketika menggunakan builder dari laravel-admin seperti grid, show dan form, kita hanya menerapkan layer service. Hal tersebut karena logika akses data sudah dikelola oleh laravel-admin.

Kode blok penggunaan laravel-admin untuk builder grid menjadi

```
> SiswaController.php
public function index(Content $content, SiswaService $siswa_srv)
{
    return $content
        ->header('Pemberkasan')
        ->description('Daftar Arsip Statis')
        ->body($siswa_srv->laravelAdminGrid());
}

> SiswaService.php
public function laravelAdminGrid()
{
    $grid = new Grid(new EloquentModel);
    $grid->column('id', __('ID'))->sortable();
    $grid->column('name', __('Nama'));
    return $grid;
}
```

Pola ini juga berlaku untuk builder lain pada laravel-admin seperti form dan show, sehingga akan ada function untuk setiap builder yang digunakan pada service. Semua function tersebut pada service **harus memiliki prefix laravelAdmin**.

Misal: laravelAdminShow, laravelAdminGrid, laravelAdminForm

6. View Resources

- ➔ Penamaan blade menggunakan **Snake Case**, yaitu semua spasi dengan garis bawah atau underscore dengan semua huruf kecil atau lowercase.
- ➔ Blade pada modul yang sama disimpan pada folder yang sama dengan nama folder sesuai modul.

```
resources
      views
            rapor
                  create_nilai_pelajaran.blade.php
                  rapor.blade.php
```

➔ Javascript pada view harus ditulis pada blade yang sama dengan memanfaatkan @section / @push / lainnya pada Laravel.

```
resources
      views
            rapor
                  rapor.blade.php
                  script.blade.php ------> X (ditulis pada blade rapor)
```

rapor.blade.php menjadi

```
> rapor.blade.php
<div class="card">
   <div class="card-body">
       IDENTITAS SISWA
   </div>
</div>

@push('script')
   <script>
   </script>
@endpush
```

atau

```
> rapor.blade.php
<div class="card">
   <div class="card-body">
       IDENTITAS SISWA
   </div>
</div>

<script>
</script>
```

➔ Tidak menyisipkan fungsi PHP atau fungsi blade pada blok algoritma javascript. Gunakan variabel untuk kasus ini dan semua variabel tersebut harus berada pada bagian paling atas pada tag script.

```
<button id="buttonSubmitComment">Kirim Komentar</button>

<script>
   $('#buttonSubmitComment').on('click', function(e) {
       e.preventDefault();
       $.ajax({
           url: '{{ route('comment.store') }}', ----> Use Variable
           type: "POST",
           dataType: 'json',
       })
   });
<script>
```

Blok kode diatas diubah ditulis menjadi

```
<button id="buttonSubmitComment">Kirim Komentar</button>

<script>
   // Transisi antara Blade dan Javascript
   const comment_store_url = '{{ route('comment.store') }}';
   const sampleData = @json($dataFromController);

   // Javascript
   $('#buttonSubmitComment').on('click', function(e) {
       e.preventDefault();
       $.ajax({
           Url: comment_store_url, ----> Gunakan Variabel
           type: "POST",
           dataType: 'json',
       })
   });
<script>
```

➔ Apabila menggunakan sub view (membuat sub element / komponen yang dipisahkan pada blade berbeda), maka blade sub view ditambahkan prefix yaitu nama blade induk dan underscore (induk_)

```
> rapor.blade.php
<div class=”col-sm-6”>
   <div id=”identitas” class="card">
       <div class="card-body"> IDENTITAS SISWA </div>
   </div>
</div>
<div class=”col-sm-6”>
   <div id=”nilai” class="card">
       <div class="card-body"> MATA PELAJARAN & NILAI </div>
   </div>
</div>
```

rapor.blade.php ingih dipisahkan menjadi sub view dengan memecah masing-masing element card, maka struktur menjadi

```
resources
      views
            rapor
                  rapor.blade.php
                  rapor_identitas.blade.php
                  rapor_nilai.blade.php
```

View dan sub view menjadi

```
> rapor.blade.php
<div class=”col-sm-6”>
   @include(‘rapor_identitas’)
</div>
<div class=”col-sm-6”>
   @include(‘rapor_nilai’)
</div>

> rapor_identitas.blade.php
<div id=”identitas” class="card">
   <div class="card-body"> IDENTITAS SISWA </div>
</div>

> rapor_nilai.blade.php
<div id=”nilai” class="card">
   <div class="card-body"> MATA PELAJARAN & NILAI </div>
</div>
```

7. Helpers

- ➔ Terdapat 2 macam helper yang digunakan yaitu berupa class dan function
- ➔ Penamaan file, class dan function dijelaskan pada poin Penamaan File, Class, Function dan Variabel
- ➔ Semua helper berupa class ditulis pada folder app/Helpers dan dapat berupa static atau non-static

Contoh penggunaan helper berupa class dengan static method

```
> app/Helpers/IndonesianTimeStatic.php
class IndonesianTimeStatic
{
   public static function toInd($tgl)
   {
       return $tgl;
   }
}

> app/Http/Controllers/TesStaticHelperController.php
class TesStaticHelperController extends Controller
{
   public function index(Request $request)
   {
       IndonesianTime::toInd();
   }
}
```

Contoh penggunaan helper berupa class dengan non-static method

```
> app/Helpers/IndonesianTime.php
class IndonesianTime
{
   public function toInd($tgl)
   {
       return $tgl;
   }
}

> app/Http/Controllers/TesHelperController.php
class TesHelperController extends Controller
{
   public function index(Request $req, IndonesianTime $timeHelper)
   {
       $timeHelper->toInd();
   }
}
```

➔ Semua helper berupa function (tanpa class) ditulis pada folder app/Helpers/Methods dan function akan di-register secara otomatis oleh dependency Helpers Register

```
> app/Helpers/Methods/time_formatter.php
if (!function_exists(to_ind)) {
   function to_ind($tgl)
   {
       return $tgl;
   }
}

> app/Http/Controllers/TesHelperController.php
class TesHelperController extends Controller
{
   public function index(Request $req)
   {
       to_ind($tgl);
   }
}
```

8. Deployment

Proses deployment menggunakan GIT dengan menambahkan remote sesuai environment pada local / developer repository.

```
demo    ssh://puspemdemo@demo.badungkab.go.id/var/git/sidumas.git (fetch)
demo    ssh://puspemdemo@demo.badungkab.go.id/var/git/sidumas.git (push)
live    ssh://sidumasadm@sidumas.badungkab.go.id/home/sidumasadm/sidumas (fetch)
live    ssh://sidumasadm@sidumas.badungkab.go.id/home/sidumasadm/sidumas (push)
origin  git@gitlab.badungkab.go.id:dev-badung/layanan-aspirasi-masyarakat-v2.git (fetch)
origin  git@gitlab.badungkab.go.id:dev-badung/layanan-aspirasi-masyarakat-v2.git (push)
```

Kemudian deployment dilakukan dengan menjalankan perintah git push.

```
git push demo master
git push live master
```

Sebelum menjalankan perintah tersebut, developer harus memastikan:

1. Repository master telah update sesuai origin terakhir (merging)
2. Modul / subsistem / perubahan telah diujicoba pada masing-masing environment sesuai poin Environment.

9. Tag on GIT Messages

Penambahan tag pada awalan pesan (message) commit pada git diatur untuk memudahkan tracing perubahan yang dilakukan pada repository, umumnya digunakan untuk pembuatan laporan pengembangan dan pemeliharaan sistem.

Berikut merupakan format pesan yang harus diikuti.

Tag	Keterangan	Contoh
DEV	Digunakan untuk commit pada project yang sedang dalam proses development awal (**belum produksi**)	#dev modul register #dev fix bug register
MAINTAIN	Digunakan untuk commit saat **pemeliharaan**, dimana project **telah berjalan produksi**. Tag ini juga digunakan saat **penambahan fitur / modul / sub-sistem yang bersifat minor** pada versi yang berjalan.	#maintain fix bug ini-itu #maintain export excel #maintain add report #maintain add multiple register feature
DEV-NG	Digunakan untuk commit saat **pengembangan next generation, versi berikutnya atau perubahan major** pada versi yang berjalan.	#dev-ng init versi 2 #dev-ng new register #dev-ng review register

10. Versioning

Versioning dilakukan setiap adanya perubahan pada aplikasi yang secara resmi telah di-release, yang mana ditandai dengan adanya commit yang di-push ke environment **Live**.

Versioning dilakukan dengan menambahkan tags pada git repository dengan kode versi yang ditulis sesuai format berikut.

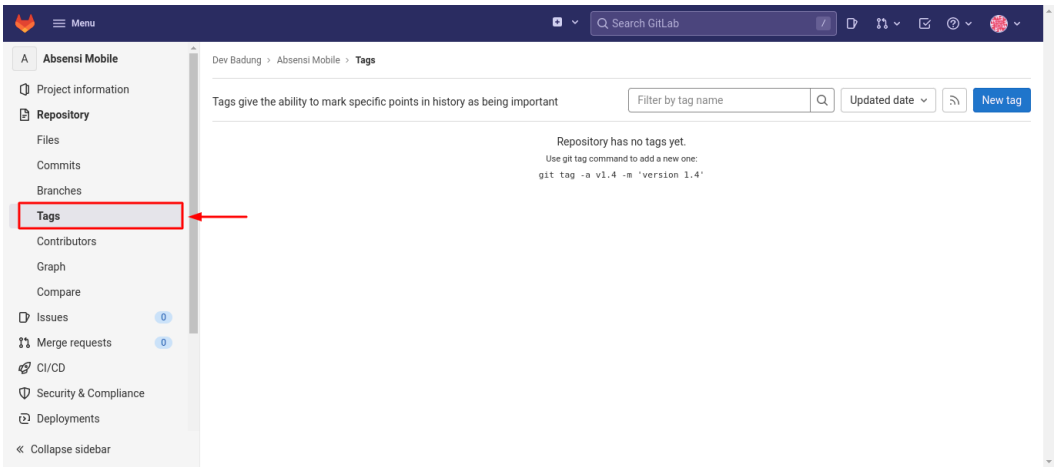
1 . 2 . 3

Keterangan:

1. Major version : Adanya perubahan besar (major changes)
2. Minor version : Adanya perubahan kecil (minor changes)
3. Patches : Bug fixes

Penambahan tags pada git repository dapat dilakukan melalui Gitlab Badung dengan tahapan sebagai berikut.

1. Pilih project
2. Pilih menu **Repository > Tags** pada halaman project



3. Isi tag name dengan kode versi sesuai format, pilih branch acuan (umumnya master), message dapat dikosongkan, dan release note dengan daftar perubahan pada aplikasi yang di-release.
4. Create tag

Berikut merupakan contoh versioning yang diterapkan:

Kode Versi	**Tanggal Release**	**Release Note**
1.0.0	1 januari 2021	First release SIDUMAS
1.0.1	11 januari 2021	Fixed bugs: - Form pengajuan - Validasi NIK
1.1.0	2 februari 2021	Fitur baru: - Verifikasi user yang boleh mengadu oleh admin - Daftar call center yang dibutuhkan masyarakat - Integrasi SILK PDAM
1.1.1	20 februari 2021	Fixed bugs di Integrasi API
1.1.2	23 februari 2021	Fixed bugs: - Foto tidak muncul pada halaman admin - Upload foto ukuran besar
2.0.0	12 desember 2022	SIDUMAS new generation. Perubahan semua alur dan mekanisme pengaduan